Machete — Parcial Butaca Cero

> Estrategia: **no escribir de cero**. Buscar algo parecido en el proyecto (Ctrl+Shift+F en VS Code), copiarlo y adaptarlo.

---

0. Comandos (desde la raíz del repo, donde está `docker-compose.yml`)

```powershell
docker compose up -d                                        # levantar
docker compose restart web                                  # reiniciar (si quedó colgado)
docker compose logs --tail 30 web                           # ver el error real
docker compose exec web python manage.py check              # ¿el código carga bien?
docker compose exec web python manage.py makemigrations     # crear migración (después de tocar models.py)
docker compose exec web python manage.py migrate            # aplicarla a la base
docker compose exec web python manage.py showmigrations     # [X] aplicada / [ ] pendiente
docker compose exec web python manage.py test web.manager # correr tests de una app
docker compose exec web python manage.py seed_demo          # cargar salas, películas y funciones de prueba (rama main)
docker compose exec web python manage.py seed_practice      # (development) salas, películas, funciones, candy, promo + 3 cuentas
docker compose exec web python manage.py seed_demo --refresh-posters
# bajar pósters reales de Wikipedia
docker compose exec web python manage.py shell              # consola de Django para probar consultas
docker compose down -v                                      # ⚠️ BORRA la base entera (después: up -d + seed)
```

Cuentas de `seed_practice` (contraseña `PracticaLocal2026!`): `gerente@local.test`, `operador@local.test`, `cliente@local.test`.

Cambiar de rama con migraciones distintas → la base queda con columnas que el código de la otra rama no conoce (`IntegrityError` al crear cosas). Lo más simple: `down -v` + `up -d` + esperar a que termine de migrar + `seed_practice`.

Después de `up -d`, esperar a que el contenedor termine de migrar antes de correr comandos (si no: `relation "..." does not exist`). Ver `showmigrations` sin `[ ]`.

Si una migración salió mal:

- Hice `makemigrations` pero NO `migrate` → borro el archivo nuevo de `migrations/`, corrijo el modelo, `makemigrations` de nuevo.
- Ya hice `migrate` → **lo más seguro:** corrijo el modelo y hago `makemigrations` + `migrate` otra vez (se crea una migración nueva encima). Si pregunta "Did you rename...?" → `y`.
- Volver atrás (avanzado): `migrate movies 0004` (vuelve la base a esa migración), después borro el archivo de la siguiente.

Cambié CSS y no se ve el cambio → Ctrl+F5 (el navegador tiene el CSS viejo en caché).

0.5 Regla de oro al copiar código

1. Antes de copiar, **entendé qué hace** el original (leelo entero).
2. Copiá la **estructura** (herencia de clase, `def post`, `get_object`, `save`, `redirect`, `<form>`...).
3. Cambiá **todo lo que tenga nombre propio del original**:
   - variables (`user` → `movie`)
   - campos (`is_active` → `is_featured`)
   - nombres de URL (`usuarios_list` → `peliculas_list`)
   - textos / mensajes
   - **la variable del `{% for %}`** del template donde pegás (`item.user` → `movie`)
4. Después de pegar: Ctrl+F del nombre viejo → no tiene que quedar ninguno.
5. Una consigna suele tener una parte **back** (datos/orden/permisos) y otra **front** (mostrar). Separarlas antes de empezar.
6. **Primero listar qué archivos voy a tocar** (modelo → migración → form → vista/URL → templates). Si la vista ya existe y usa ese form/template, no hace falta tocar vista ni URL.

---

1. Mapa del proyecto: ¿dónde va cada cosa?

Qué	Dónde
Modelos (datos)	`django/domain/<app>/models.py`
Admin	`django/domain/<app>/admin.py`
Pantallas del **gerente/operador**	`django/web/manager/` (views.py, forms.py, urls.py, templates/)
Pantallas del **cliente** (cartelera, detalle)	`django/web/catalog/`
Compra de entradas, candy, mis entradas	`django/web/tickets/`
Configuración global del cine	`django/domain/cinema/models.py` → `CinemaSettings`
Estilos del cliente (cartelera, detalle, candy)	`django/web/catalog/static/catalog.css`

Archivos que aparecen seguido:

Qué	Archivo
Form de funciones (manager)	`web/manager/forms.py` → `ScreeningForm`
Listado de funciones (manager)	`web/manager/templates/manager_funciones_list.html`
Detalle de función (cliente)	`web/catalog/templates/screening_detail.html`
Tarjeta de producto/combo de candy (cliente)	`web/tickets/templates/_candy_item.html` (**se usa para productos Y combos**)
Productos y combos de candy	`domain/candy/models.py` → `CandyProduct`, `CandyCombo`

En `development` las vistas son carpetas, no un solo archivo:

Qué	Dónde
Vistas del manager	`web/manager/views/` → `home.py` (dashboard, config), `movies.py`, `base.py` (`ManagerListView`)...

Qué	Dónde
Vistas de tickets	`web/tickets/views/` → `purchases.py` (mis entradas)...
Exportar la vista nueva	`views/__init__.py` → `from web.manager.views.home import MiVista as MiVista` (si no, `urls.py` no la encuentra)

Imports del proyecto empiezan con `domain.` o `web.` — **nunca** `django.domain...`

```python
from domain.movies.models import Movie
from domain.cinema.models import CinemaSettings
```

---

2. Recetas: "quiero hacer X → copiá de Y"

2.1 Agregar un campo a un modelo existente (ej: trailer, director, aviso)

1. `models.py` → agregar campo. **Copiar de:** cualquier campo de `Movie` / `Screening`.
2. `makemigrations` + `migrate`.
3. `admin.py` → agregar a `fields` (si la clase admin tiene `fields`).
4. `web/manager/forms.py` → agregar a `Meta.fields` y `labels` del form (ej: `MovieForm`, `ScreeningForm`).
5. Template → mostrarlo: `{{ movie.campo }}`.

Tipos de campo frecuentes:

```python
models.CharField(max_length=100, blank=True)          # texto corto (opcional)
models.TextField(blank=True)                          # texto largo
models.URLField(blank=True)                           # link
models.PositiveIntegerField(default=10)               # número
models.BooleanField(default=False)                    # sí/no
models.DateTimeField(auto_now_add=True)               # fecha de creación
automática
models.ForeignKey(Movie, on_delete=models.CASCADE, related_name="reviews")
models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)   # usuario
```

- **Campo nuevo obligatorio en tabla con datos → necesita `default=`** (o `blank=True` si es texto).
- `blank=True` = opcional en formularios. En textos NO poner `null=True`.

- `CharField(blank=True)` sin `default` → las filas que ya existen quedan con `""` y la migración no pregunta nada.
- `max_length=80` ya valida el máximo en el form → **no hace falta** `clean_` para el máximo.
- `verbose_name="Aviso para el público"` → el form usa ese texto como etiqueta (si no, muestra el nombre del campo en inglés, ej: "Warning"). Alternativa: ponerlo en `labels` del form.

```python
warning = models.CharField(max_length=80, blank=True, verbose_name="Aviso para el público", db_column="aviso")
```

- Con `verbose_name` en el modelo, `labels` en el form sobra (son dos formas de lo mismo).

`blank` vs `null` (campos opcionales):

	Qué controla	Texto opcional	Fecha / número / FK opcional
`blank=True`	el **formulario** (se puede dejar vacío)	✅	✅
`null=True`	la **base de datos** (la columna acepta "nada")	❌ (vacío = `""`)	✅ (no existe una "fecha vacía")

```python
models.CharField(max_length=80, blank=True)       # texto opcional
models.DateField(null=True, blank=True)           # fecha opcional → LAS DOS
```

- Fecha con solo `blank=True` → `makemigrations` pregunta *"It is impossible to add a non-nullable field..."* → elegir **2 (Quit)**, agregar `null=True`, correr de nuevo.
- Fecha con solo `null=True` → la base acepta vacío pero **el form la sigue exigiendo** ("Este campo es obligatorio").
- Un campo opcional sin cargar vale `None` en Python → chequear antes de comparar (`if self.fecha and ...`).

2.2 Campo con opciones fijas (ej: idioma Doblada/Subtitulada)

Copiar de: `Movie.Genre` / `Movie.Rating` / `Screening.Status`.

```python
class Language(models.TextChoices):
```

```
    DUBBED = "doblada", "Doblada"          # valor (se guarda) , etiqueta (se
muestra)
    SUBTITLED = "subtitulada", "Subtitulada"

language = models.CharField(max_length=20, choices=Language.choices,
default=Language.SUBTITLED)
```

En templates:

- Mostrar al usuario → `{{ obj.get_language_display }}` (existe solo si el campo tiene `choices=`)
- Comparar → `{% if obj.language == "doblada" %}`

Buscar ejemplos: Ctrl+Shift+F → `_display`

Pasos completos (la hice sola en ~30 min, ver feature 15):

1. `class Language(models.TextChoices)` **adentro** del modelo, al lado de `Status` (en `Screening`).
2. Campo con `choices=` + `default=` (el `default` cubre las filas que ya existen) + `db_column` en castellano.
3. `makemigrations` + `migrate`.
4. Form: agregar a `Meta.fields` + `labels`. Se dibuja solo como `<select>` con las opciones.
5. Templates: `get_..._display` en **todos** los lugares donde se muestra (manager y cliente).

- Con `default=`, el `<select>` **no tiene opción vacía** (siempre hay una elegida).
- `self.fields["x"].empty_label = "..."` sirve **solo para campos `ForeignKey`** (`movie`, `room`). En un campo con `choices` no da error pero **no hace nada** → no ponerlo.

2.3 Valor configurable por el gerente (ej: máximo de entradas por compra, umbral de stock bajo)

Copiar de: `candy_max_item_quantity`, `ticket_reprogram_minutes` o `candy_low_stock_threshold`.

1. `domain/cinema/models.py` → campo en `CinemaSettings` (con `default=`).
2. `makemigrations` + `migrate`.
3. `web/manager/forms.py` → `CinemaSettingsForm`: campo declarado (`forms.IntegerField(min_value=1, label=..., help_text=...)`) + agregarlo a `Meta.fields`.
    - Ojo: el `min_value` del form es el que manda en la pantalla. Revisar que coincida con la consigna (ej: "mínimo 1" → `min_value=1`).

4. Usarlo en el back: `CinemaSettings.current().mi_campo`
5. Usarlo en el front: `{{ cinema.mi_campo }}` (casi todas las vistas pasan `cinema` al contexto).

Reemplazar una constante fija por un valor configurable (ej: `LOW_STOCK_THRESHOLD = 10`):

- Buscar la constante con Ctrl+Shift+F → reemplazar cada uso por `CinemaSettings.current().mi_campo`.
- Si se usa dentro de una `@property` del modelo, el cambio se refleja **enseguida** en todas las pantallas que usan esa property (manager y cliente), sin tocar las vistas.

```python
@property
def is_low_stock(self):
    return 0 < self.stock <= CinemaSettings.current().candy_low_stock_threshold
```

- En templates **no** se puede llamar `CinemaSettings.current()` → usar una property del modelo (`{% if item.is_low_stock %}`) o `{{ cinema.mi_campo }}`.

2.4 Validar un campo de un form

Copiar de: `MovieForm.clean_image` (manager/forms.py), `ScreeningForm.clean_starts_at`.

```python
def clean_<campo>(self):
    valor = self.cleaned_data.get("<campo>")
    if <no cumple>:
        raise forms.ValidationError("Mensaje de error.")
    return valor                        # ¡SIEMPRE devolver!
```

- Validar varios campos juntos → `def clean(self):` (ver `ScreeningForm.clean`).
- Si el tipo de campo ya valida (`URLField`, `EmailField`, `IntegerField(min_value=1)`, `max_length`) → no hace falta `clean_`.
- **Campo opcional con mínimo** ("si se carga, al menos N"): validar **solo si vino algo** (`if valor and ...`), así el vacío sigue siendo válido. Largo de un texto → `len(valor)`.
- El método va **dentro de la clase del form**, al mismo nivel que `class Meta` (no adentro de `Meta`), y se llama `clean_` + nombre exacto del campo.
- Usar `self.cleaned_data.get("campo")` (no `["campo"]`): si el campo vino vacío da `None` en vez de explotar.
- Validar fechas ("no más de un año en el futuro", "no en el pasado") → comparar contra `timezone.localdate()` ± `timedelta` (ver 2.15). Si el campo es opcional, primero chequear que no sea `None`.

2.5 Página nueva (vista + URL + template + link en menú) — ej: "Funciones de hoy"

Copiar de: `TodayScreeningView` (manager/views.py) o `DashboardView`.

```python
class MiVista(StaffAccessMixin, TemplateView):          # o ManagerRequiredMixin
    template_name = "manager_mi_pagina.html"            # ¡mismo nombre que el archivo!

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["section"] = "Mi sección"
        context["items"] = Modelo.objects.filter(...)
        return context
```

- `urls.py`: `path("mi-pagina/", views.MiVista.as_view(), name="mi_pagina"),`
- Template nuevo en `web/manager/templates/`:

```html
{% extends "manager_base.html" %}
{% block content %}
  ...
{% endblock %}
```

- Link en el menú: `web/manager/templates/manager_base.html` → `<a href="{% url 'manager:mi_pagina' %}">Texto</a>`

Permisos (Sprint 3):

Mixin	Quién entra
`StaffAccessMixin`	cualquier staff (gerente, operador)
`ManagerRequiredMixin`	solo gerente
`@login_required` (vistas función)	cualquier usuario logueado

2.6 Listado del manager con búsqueda y orden (ABM)

Copiar de: `MoviesListView` / `MovieCreateView` / `MovieUpdateView` / `MovieDeleteView` + sus 4 URLs + `manager_peliculas_list.html`.

- Los forms de alta/edición usan el template genérico `manager_form.html` (no hay que hacer template).

- Búsqueda: método `apply_search` con `Q(...) | Q(...)`:

```python
queryset.filter(Q(title__icontains=texto) | Q(director__icontains=texto))
```

- Columna nueva en tabla: `<th>` + `<td>` + actualizar `colspan` del `{% empty %}`.

2.7 Acción con botón que hace POST (activar/desactivar, destacar, publicar...)

Copiar de: `UserToggleActiveView` o `MovieToggleFeatureView` (manager/views.py) + su form en `manager_usuarios_list.html` / `manager_peliculas_list.html`.

1. Modelo: `is_featured = models.BooleanField(default=False)` → `makemigrations` + `migrate`.
2. Vista:

```python
class MovieToggleFeatureView(ManagerRequiredMixin, DetailView):   # mixin = quién puede; DetailView = 1 objeto por <pk>
    model = Movie                                                 # de qué tabla busca

    def post(self, request, *args, **kwargs):                     # solo se ejecuta con POST
        movie = self.get_object()                                 # busca por el pk de la URL (404 si no existe)
        movie.is_featured = not movie.is_featured                 # toggle: invierte True/False
        movie.save(update_fields=["is_featured"])                 # guarda solo esa columna
        estado = "destacada" if movie.is_featured else "sin destacar"
        messages.success(request, f"La película quedó {estado}.")
        return redirect("manager:peliculas_list")                 # NOMBRE de la URL, no la ruta
```

3. URL: `path("peliculas/<int:pk>/destacar/", views.MovieToggleFeatureView.as_view(), name="peliculas_feature"),`
4. Template (botón que cambia de texto según el estado):

```html
<form method="post" action="{% url 'manager:peliculas_feature' movie.pk %}">
  {% csrf_token %}
  <button type="submit">
```

```
    {% if movie.is_featured %}Quitar destacado{% else %}Destacar{% endif %}
  </button>
</form>
```

- **`<a href>` = GET** (solo navegar). **Cambiar datos = POST** → `<form method="post">` + `{% csrf_token %}`.
- `redirect("app:nombre")` usa el `name=` de urls.py. `redirect("manager:peliculas/")` ❌.
- El texto del botón dice lo que **va a hacer**: si ya está destacada → "Quitar destacado".

2.8 Modelo nuevo completo con form del cliente (ej: reseñas)

Copiar de: `Review`, `ReviewForm` (web/catalog/forms.py), `movie_review_view` (web/catalog/views.py).

1. Modelo con ForeignKey(s) → `makemigrations` + `migrate`.
2. `admin.py` → `admin.site.register(Modelo)` (**una sola vez** por modelo).
3. `ModelForm` con `fields = (...)`.
4. Vista que muestra la página → agregar al `render(...)`: `"form": MiForm()`, datos a mostrar.
5. Vista que recibe el POST:

```python
@login_required
@require_POST
def mi_post_view(request, pk):
    obj = get_object_or_404(Movie, pk=pk)
    form = MiForm(request.POST)
    if form.is_valid():
        Modelo.objects.create(movie=obj, user=request.user, **form.cleaned_data)
        # o update_or_create(movie=obj, user=request.user,
defaults=form.cleaned_data)
        messages.success(request, "OK")
    else:
        messages.error(request, "Error")
    return redirect("movie_detail", pk=obj.pk)      # después de un POST, siempre
redirect
```

6. URL nueva.
7. Template: `<form method="post" action="{% url '...' obj.pk %}">{% csrf_token %}{{ form.as_p }}<button type="submit">Enviar</button></form>`

Una sola fila por usuario+película:

```python
class Meta:
    constraints = [models.UniqueConstraint(fields=["movie", "user"],
name="unico_por_usuario")]
```

2.9 Filtrar con parámetros de la URL (ej: `/?genero=ACCION`)

Copiar de: `home_view` (catalog/views.py) o `DashboardView` (`self.request.GET.get(...)`).

```python
valor = request.GET.get("genero", "")
queryset = Movie.objects.all()
if valor:
    queryset = queryset.filter(genre=valor)
```

Template: `<a href="?genero={{ value }}">{{ label }}</a>` y `{% for value, label in genres %}` con `Movie.Genre.choices`.

2.10 Ordenar un listado (ej: destacadas primero)

Copiar de: `ordering_options` de `MoviesListView` (manager/views.py) o el `order_by` de `home_view`.

```python
queryset = queryset.order_by("-is_featured", "title")   # antes del list(queryset)
```

- `-` = descendente (mayor a menor). Sin `-` = ascendente.
- Booleanos: `False` = 0, `True` = 1 → `"-campo_bool"` pone los `True` primero.
- Segundo campo = desempate dentro de cada grupo.
- Patrón de encadenar: `queryset = queryset.filter(...)` → `queryset = queryset.order_by(...)` → `movies = list(queryset)`.

2.11 Marca/etiqueta condicional en una tarjeta o fila

Copiar de: la etiqueta de promo en `home.html` (`{% if movie.promotion_quote %}<span class="movie-card-promo">...`) o los `status-pill` de los listados del manager.

```html
{% if movie.is_featured %}<span class="movie-card-promo">⭐ Destacada</span>{%
endif %}
```

Va **dentro** del `{% for %}`, usando la variable del loop.

- **Texto opcional**: envolver todo (la caja y el texto) en `{% if obj.campo %}` → un texto vacío `""` es falso, así que no se muestra nada (ni caja vacía ni "None").

Estado con varias opciones excluyentes (ej: comprable / últimas unidades / agotado): pensarlo como **decisiones separadas**, no como un `if/elif` largo.

```
HTML
{% if item.is_low_stock %}<span class="candy-last-units">¡Últimas
unidades!</span>{% endif %}   {# 1) ¿cartel extra? #}
{% if item.max_quantity > 0 %}
{# 2) ¿se puede comprar? #}
  ...botones + / - ...
{% else %}
  <span class="candy-sold-out">Agotado</span>
{% endif %}
```

- Un cartel de aviso **se suma** a los botones, no los reemplaza (el producto con stock bajo se tiene que poder seguir comprando).
- Revisar los **bordes**: si la property usa `<=`, el template no puede usar `<` (con stock == umbral no entraba en ninguna rama → salía "Agotado").
- Comparar contra el dato correcto: `max_quantity` = mínimo entre stock y límite por compra; **no** es el stock.

2.12 Páginas del cliente (catalog / tickets)

- En `development` usan `{% extends "client_base.html" %}` + `{% block content %}<main class="client-main">...</main>{% endblock %}` (ver `my_tickets.html`). En la rama vieja eran HTML completos con `{% include "_client_header.html" %}`: **mirar el primer renglón de un template vecino** antes de arrancar.
- Para una página nueva del cliente: **copiar un template entero** parecido (ej: `my_tickets.html`) y vaciar el `<main>`.
- El nombre de las URLs del cliente va **sin prefijo** (`{% url 'movie_detail' pk %}`, `redirect("my_tickets")`): `catalog` y `tickets` no tienen `app_name`. El manager sí (`manager:`).
- Vistas del cliente = funciones con `@login_required` (copiar de `my_tickets_view`).
- Links del menú del cliente: `web/catalog/templates/_client_header.html`.

2.13 Template compartido por dos modelos (ej: `_candy_item.html` para productos Y combos)

- Si agrego una property/método **solo a un modelo**, en el otro esa variable **no existe → queda vacía, sin error**.
- Consecuencias:

    - `{% if item.prop_que_no_existe %}` → siempre falso (a veces sirve: el cartel simplemente no aparece en combos).
    - `{% if item.numero > item.prop_que_no_existe %}` → compara número contra `""` → **falso en silencio** → cae en el `else` (así los combos salían "Agotado").
- Solución: usar properties que existan en **los dos** modelos (`available_stock`, `max_quantity`), o una property booleana que en el otro modelo dé falso a propósito.
- Antes de tocar un template con `_` adelante (`_algo.html` = parcial incluido), buscar con Ctrl+Shift+F dónde se incluye.

2.14 Arreglar un layout solo con CSS (sin tocar el HTML)

Problema típico: una fila `display: flex` pensada para 2 elementos (precio | botones) y le agregué un 3.º (cartel) → no entra.

```css
.candy-item-footer {
  flex-wrap: wrap;      /* permite bajar elementos a otra línea */
}

.candy-last-units {
  flex-basis: 100%;     /* el cartel ocupa toda la fila */
  order: -1;            /* va primero (arriba). order: 1 → abajo */
}
```

- Los elementos que no tienen el cartel no cambian (siguen en una sola línea).
- Buscar la clase en `catalog.css` con Ctrl+F antes de inventar una nueva.
- Después de guardar → **Ctrl+F5**.

2.15 Cuentas con fechas ("últimos 7 días", "dentro de un año")

Copiar de: `web/manager/dashboard.py` (`today - timedelta(days=days - 1)`), `Promotion` en `domain/promotions/models.py` (property que usa `timezone.localdate()`), `Screening.is_editable`.

```python
from datetime import timedelta
from django.utils import timezone

timezone.localdate()                        # HOY, solo fecha  → para comparar con
DateField
timezone.now()                              # AHORA, fecha+hora → para comparar
con DateTimeField
timezone.localdate() - timedelta(days=6)    # la FECHA de hace 6 días
timezone.localdate() + timedelta(days=365)  # la FECHA de dentro de un año
```

- `timezone` es un **módulo**: `timezone.localdate()` ✅ — `timezone().localdate()` ❌ (`'module' object is not callable`).
- `timedelta(days=6)` es una **duración**, no una fecha. Sola no se puede comparar con una fecha: hay que sumarla o restarla a una fecha.
- **"Últimos 7 días, hoy incluido"** = desde `hoy - 6 días` hasta `hoy` (04, 05, 06, 07, 08, 09, 10 = 7 días). Con `- 7` serían 8.
- "Está entre dos fechas" → comparación encadenada, leyéndola como una línea de tiempo:

```python
desde <= self.fecha <= hoy        # hace 6 días ... fecha ... hoy
```

- Leer la comparación en voz alta: `hoy <= fecha` = "hoy es anterior a la fecha" = fecha **futura**.
- `DateField` se compara con `localdate()`; `DateTimeField` con `now()`. Mezclarlos da `TypeError`.

La cuenta va en Python (property del modelo), no en el template:

```python
@property
def is_algo_reciente(self):                    # nombre que diga qué devuelve: is_..., has_...
    if not self.fecha:                         # campo opcional → puede ser None
        return False
    hoy = timezone.localdate()
    return hoy - timedelta(days=6) <= self.fecha <= hoy
```

Template: `{% if obj.is_algo_reciente %}...{% endif %}` — una sola condición, sin `{% if obj.fecha %}` afuera porque la property ya lo chequea.

- "Marcar" ≠ "filtrar": si la consigna dice que algunas muestran una marca, **todas se siguen listando** → property + `{% if %}`, no `.filter()` en la vista.
- Mostrar una fecha: `{{ obj.fecha|date:"d/m/Y" }}`. Si puede estar vacía: `{{ obj.fecha|date:"d/m/Y"|default:"—" }}`.

2.16 Cómo se dibuja un campo en el form: campo vs widget

En Django hay dos familias de clases con nombres parecidos:

Familia	Ejemplos	Qué hace	Dónde va
Campo	`forms.DateField`, `forms.IntegerField`, `forms.CharField`	**valida** el dato	declarado en la clase del form

Familia	Ejemplos	Qué hace	Dónde va
Widget	`forms.DateInput`, `forms.TextInput`, `forms.TimeInput`, `forms.ClearableFileInput`	**dibuja** el `<input>` en el HTML	en `Meta.widgets` o `widget=`

- En `Meta.widgets` solo van widgets (terminan en `Input`). Poner un campo ahí → `TypeError: Field.__init__() got an unexpected keyword argument 'attrs'` y **el servidor se cae**.
- Un `DateField` por defecto se dibuja como `<input type="text">` (parece texto libre, pero al enviar valida que sea una fecha). Para que aparezca el calendario:

```python
widgets = {
    "release_date": forms.DateInput(attrs={"type": "date"}, format="%Y-%m-%d"),
}
```

- `attrs={"type": ...}` = tipo de input de **HTML** (`"date"`, `"time"`, `"color"`, `"number"`), no el nombre del campo.
- `format="%Y-%m-%d"` → al **editar**, la fecha guardada aparece cargada. Sin eso (o con el formato mal) el campo aparece vacío.
- Mayúsculas y minúsculas importan: `%Y` año, `%m` mes, `%d` día — `%M` son **minutos**, `%D` es otra cosa.
- **Copiar de:** `TimeInput(attrs={"type": "time"}, format="%H:%M")` y `TextInput(attrs={"type": "color"})` en manager/forms.py.
- Es estética: si la consigna pide solo lo funcional, no es obligatorio.

2.17 Reporte con totales por fila (ej: ranking de ventas por película) — `annotate`

Copiar de: `MoviesListView` (manager/views/movies.py) para la tabla + `RankingView` (manager/views/home.py).

`annotate` vs `aggregate`:

	Devuelve	Ejemplo
`aggregate(...)`	**un solo número** para toda la tabla	"entradas vendidas hoy" (KPI)
`annotate(...)`	**un campo extra en cada fila**	"entradas vendidas de cada película" (ranking)

```python
Room.objects.annotate(cantidad_butacas=Count("seats"))   # a cada sala le pega su cantidad de butacas
sala.cantidad_butacas                                    # se usa como un campo más
.order_by("-cantidad_butacas")                           # y se puede ORDENAR por él
```

- `Count("relacion")` **cuenta** filas relacionadas. `Sum("relacion__campo")` **suma** un campo.
- Dentro va el **`related_name`**, el mismo que en `movie.screenings.all()`.

Cómo armar el camino (`__`) hasta el dato:

1. ¿Qué número quiero? → buscar el modelo y el campo (entradas → `TicketPurchase.quantity`).
2. ¿Cómo llego desde mi modelo? → seguir las `ForeignKey` **para atrás**, usando el `related_name` de cada una:
   - `Screening.movie = ForeignKey(Movie, related_name="screenings")` → desde Movie: `screenings`
   - `TicketPurchase.screening = ForeignKey(Screening, related_name="ticket_purchases")` → desde Screening: `ticket_purchases`
3. Unir con doble guion bajo: `Movie → screenings → ticket_purchases → quantity`
4. Buscar quién apunta a un modelo: Ctrl+Shift+F → `ForeignKey(Screening`

```python
from decimal import Decimal
from django.db.models import Count, Sum
from django.db.models.functions import Coalesce

def get_queryset(self):
    queryset = Movie.objects.annotate(
        cantidad_funciones=Count("screenings", distinct=True),
        entradas_vendidas=Coalesce(Sum("screenings__ticket_purchases__quantity"), 0),
        recaudacion=Coalesce(Sum("screenings__ticket_purchases__total"), Decimal("0")),
    )
    search_query = self.get_search_query()
    if search_query:
        queryset = self.apply_search(queryset, search_query)
    return self.apply_ordering(queryset)
```

Template: `{{ movie.cantidad_funciones }}`, `{{ movie.entradas_vendidas }}`, `{{ movie.recaudacion }}`.

Reglas que aprendí a los golpes:

- **`distinct=True` solo en el `Count`**: al juntar funciones con compras, cada función se repite una vez por compra → sin `distinct` el conteo se infla. En un **`Sum` NO**: `Sum(distinct=True)` suma solo los valores distintos (dos compras de 2 entradas → da 2 en vez de 4).
- **`Sum` sin filas da `None`**, no 0 → envolver en `Coalesce(..., 0)`. `Count` nunca da `None` (no lleva `Coalesce`).
- **`None` rompe el orden**: PostgreSQL pone los `NULL` **primero** en orden descendente → las películas sin ventas aparecían arriba de todo en "Más recaudación". Con `Coalesce` pasan a ser 0 y se ordenan bien.
- **El tipo del `Coalesce` tiene que coincidir con el campo:** entero (`quantity`, `Count`) → `0`; decimal (`total`, precios) → `Decimal("0")`. Mezclar → `Expression contains mixed types`.
- **El `annotate` va ANTES del `order_by`**: no se puede ordenar por un campo que todavía no existe. Por eso reescribí `get_queryset` (el de `ManagerListView` ya ordena adentro) en vez de hacer `super().get_queryset().annotate(...)`.
- Ordenar por el campo calculado: agregarlo **primero** en `ordering_options` para que sea el orden por defecto: `("recaudacion_desc", "Más recaudación", ("-recaudacion", "title"))`.
- `movie.screenings.count` en el template también muestra el número, pero hace una consulta por fila y **no se puede ordenar por eso**.
- **Filtro por fechas sin perder las filas en 0**: `.filter(screenings__ticket_purchases__created_at__date__gte=...)` **saca** las películas sin ventas en ese rango. Para que queden con 0, la condición va **adentro** del `Sum`: `Sum("...", filter=Q(...))`.
- Probar consultas en el `shell` antes de pasarlas a la vista.

`TemplateView` vs `ListView`:

	Qué le da al template	**Cuándo**
`TemplateView`	**nada** automático: solo lo que yo cargo en `get_context_data`	páginas armadas a mano (dashboard)
`ListView` / `ManagerListView`	`object_list` (+ paginación, búsqueda y orden) usando `model` / `get_queryset`	tablas / listados

- Copié el HTML de un `ListView` (`{% for movie in object_list %}`) a una vista `TemplateView` → `object_list` no existía → tabla vacía **sin error**.

- Atributos como `model`, `ordering_options`, `apply_search` **no hacen nada** si la clase no hereda de `ManagerListView`: son configuración que lee esa clase.
- `ManagerListView` ya incluye `ManagerRequiredMixin` (no repetirlo).
- `_list_controls.html` y `_pagination.html` solo funcionan con `ManagerListView`.
- `section = "Ranking"` tiene que coincidir con el `{% if section == 'Ranking' %}` del link en `manager_base.html` (si no, se resalta otra sección del menú).

2.18 Marcar / desmarcar por usuario (ej: películas favoritas) — modelo "relación"

Copiar de: 2.8 (reseñas) para el modelo + 2.7 para el botón POST.

Diferencia con 2.7: ahí el toggle cambia un **booleano de la película** (es igual para todos). Acá cada **usuario** tiene lo suyo → hace falta un **modelo nuevo** que una usuario + película. "Marcar" = **crear la fila**, "desmarcar" = **borrarla**.

1. **Modelo** (al final de `domain/movies/models.py`, **pegado a la izquierda**, no adentro de `class Movie`):

```python
from django.conf import settings

class Favorite(models.Model):
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
related_name="favorites")
    movie = models.ForeignKey(Movie, on_delete=models.CASCADE,
related_name="favorited_by")
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        constraints = [
            models.UniqueConstraint(fields=["user", "movie"],
name="unique_favorite_per_user"),
        ]
```

   - `UniqueConstraint` → **la base** rechaza duplicados (no solo la vista).
   - `auto_now_add=True` → fecha de creación sola → sirve para "la más reciente primero".
   - **No** importar `Movie` en su propio archivo (`from domain.movies.models import Movie` dentro de `domain/movies/models.py` ❌): ya está definida ahí.
2. **Vista toggle** (`web/catalog/views.py`):

```python
from django.views.decorators.http import require_POST
```

```
@login_required          # no logueado → al login
@require_POST            # GET → 405
def toggle_favorite_view(request, pk):
    movie = get_object_or_404(Movie, pk=pk)
    favorite = Favorite.objects.filter(user=request.user, movie=movie).first()   #
None si no existe
    if favorite:
        favorite.delete()
    else:
        Favorite.objects.create(user=request.user, movie=movie)
    return redirect("movie_detail", pk=pk)
```

URL: `path("peliculas/<int:pk>/favorita/", views.toggle_favorite_view, name="toggle_favorite"),`

3. **Saber si ya está marcada** (en la vista del detalle, y pasarlo al `render`):

```
Python
is_favorite = request.user.is_authenticated and
Favorite.objects.filter(user=request.user, movie=movie).exists()
```

4. **Botón** (solo si está logueado):

```
HTML
{% if request.user.is_authenticated %}
  <form method="post" action="{% url 'toggle_favorite' movie.pk %}">
    {% csrf_token %}
    <button type="submit">{% if is_favorite %}♥ Quitar de favoritas{% else %}♡
Agregar a favoritas{% endif %}</button>
  </form>
{% endif %}
```

5. **Página "Mis favoritas"**:

```
Python
@login_required
def my_favorites(request):
    favorites =
Favorite.objects.filter(user=request.user).select_related("movie").order_by("-crea
ted_at")
    return render(request, "my_favorites.html", {"cinema":
CinemaSettings.current(), "favorites": favorites,
                                                "manager_role":
manager_role(request.user)})
```

- `filter(user=request.user)` → cada uno ve **solo lo suyo**.
- `select_related("movie")` → trae la película en la misma consulta (en el template: `{{ favorite.movie.title }}`).
- Vista nueva = **función nueva y corta**. Copiar `my_tickets` entera y dejarle la lógica de compras no sirve: hay que reemplazar el cuerpo.

Cómo funciona `{% url 'toggle_favorite' movie.pk %}`:

- La ruta tiene un **hueco** `<int:pk>` → `movie.pk` es el número que va en ese hueco.
- Wicked (pk 4) → `action="/peliculas/4/favorita/"` → Django saca el `4` de la URL y llama `toggle_favorite_view(request, pk=4)`.
- Si la ruta tiene `<int:pk>` y no le paso nada → `NoReverseMatch ... with no arguments`.
- Antes de usar un `{% url %}`, abrir `urls.py` y ver: ¿el `name` es el de **mi** acción? ¿cuántos `<...>` tiene la ruta?

---

3. Consultas (ORM) que más se usan

Python

```
Modelo.objects.all()
Modelo.objects.filter(campo=valor)                          # igual
Modelo.objects.filter(titulo__icontains="abc")              # contiene (sin
mayúsculas)
Modelo.objects.filter(starts_at__date=timezone.localdate()) # de hoy
Modelo.objects.filter(precio__gte=100)                      # >=  (gt, lt, lte)
Modelo.objects.exclude(...)
.order_by("campo") / .order_by("-campo")                    # asc / desc
.select_related("movie", "room")                            # traer FKs en la
misma consulta
.count()
.exists()
.aggregate(total=Sum("quantity"))["total"] or 0             # suma (None si no hay
filas)
.aggregate(avg=Avg("stars"))["avg"]                         # promedio
get_object_or_404(Modelo, pk=pk)
movie.reviews.all()                                         # relación inversa
(related_name)
.annotate(n=Count("screenings", distinct=True))             # campo calculado POR
FILA (ver 2.17)
.annotate(t=Coalesce(Sum("screenings__ticket_purchases__total"), Decimal("0")))
Modelo.objects.filter(...).first()                          # el primero o None
(sin error si no hay)
Modelo.objects.create(campo=valor, ...)                     # crear y guardar
obj.delete()                                                # borrar
```

Imports: `from django.db.models import Q, Sum, Avg, Count` — `from django.db.models.functions import Coalesce` — `from decimal import Decimal`

---

4. Templates (sintaxis)

```HTML
{{ variable }}   {{ obj.campo }}   {{ obj.fk.campo }}
{% if cond %} ... {% elif otra %} ... {% else %} ... {% endif %}
{% for x in lista %} ... {% empty %} (si está vacía) {% endfor %}
{% url 'nombre' %}   {% url 'manager:nombre' obj.pk %}
{% csrf_token %}                                  (en todo form POST)
{{ fecha|date:"d/m/Y H:i" }}   {{ numero|floatformat:1 }}   {{ x|default:"—" }}
{% if user.is_authenticated %}                    (usuario logueado)
{% extends "manager_base.html" %} {% block content %}{% endblock %}
```

- Links que abren pestaña nueva: `<a href="..." target="_blank" rel="noopener">`
- **Cerrar todas las etiquetas** (`</a>`, `</div>`).
- ⚠️ **Variable inexistente en un template = vacía, SIN error.**
  - En `{% if item.movie.is_featured %}` → siempre falso, falla en silencio.
  - En `{% url '...' item.movie.pk %}` → explota con `NoReverseMatch ... arguments '('',)'`.
  - En `{% if item.numero > item.no_existe %}` → compara contra `""` → falso en silencio.
  - Solución: usar el nombre del `{% for X in ... %}` de ESE archivo, y propiedades que existan en ese modelo.
- En templates **no** se pueden llamar funciones con argumentos (ej: `CinemaSettings.current()`). Usar properties del modelo o variables que pase la vista.
- En templates **no hay Python**: nada de `timedelta(...)`, `timezone...`, cuentas ni funciones propias. Toda cuenta va en una `@property` del modelo o en la vista; el template solo pregunta `{% if obj.property %}`.
- Texto opcional pegado a otros con separador (`A · B · {{ obj.aviso }}`): meter **el separador adentro del `{% if %}`**, si no queda un " · " colgando cuando está vacío.
- Al copiar una celda/condición de otra columna, revisar que el `{% if %}` pregunte por **el campo nuevo** y no por el del original (ej: quedó `{% if movie.is_featured %}` para mostrar la fecha de estreno).
- **Un mismo dato puede dibujarse en dos ramas** de un `{% if %}...{% else %}` (ej: chip de horario **disponible** vs **agotado** en `screening_detail.html`). Si la consigna dice "cada horario", agregarlo en **las dos** ramas.

5. Errores que ya me pasaron y cómo se arreglan

Síntoma	Causa	Arreglo
`column ... does not exist`	Faltó aplicar migración	`makemigrations` + `migrate` (ver `showmigrations`)
`ERR_EMPTY_RESPONSE` / página no carga	Servidor caído por error en un `.py`	`docker compose logs --tail 30 web` → leer la **última línea** y el `File "..." line N`
`ModuleNotFoundError: No module named 'django.domain'`	Import mal	`from domain.xxx.models import ...`
`TemplateDoesNotExist`	`template_name` ≠ nombre del archivo, o archivo fuera de `templates/`	Igualar nombres. Mirar "Template-loader postmortem"
`AlreadyRegistered`	Modelo registrado 2 veces en admin	Dejar un solo `@admin.register` / `admin.site.register`
`must be a list or tuple`	`("x")` no es tupla	`("x",)` con coma
Dato no aparece en el template	Lo pasé en la vista equivocada	Ver qué vista renderiza ese template (urls.py → vista → `render`)
403 al enviar un form	Falta `{% csrf_token %}`	Agregarlo dentro del `<form>`
`NoReverseMatch`	Nombre de URL mal escrito o falta el `pk`	Revisar `name=` en urls.py y el prefijo `manager:`
`NoReverseMatch ... with arguments '('',)'`	La URL existe pero el `pk` llegó vacío: variable mal nombrada en el template	Usar la variable del `{% for %}` del archivo (ej: `movie.pk`, no `item.movie.pk`)
El botón/dato no cambia en el front, pero el mensaje de éxito sale	El back anda; el `{% if %}` usa una variable que no existe	Ctrl+F del nombre viejo en el template

Síntoma	Causa	Arreglo
`class MiVista():` no hace nada	Clase sin herencia	`(ManagerRequiredMixin, DetailView)` etc.
Toca un link y no cambia nada / 405	Usé `<a>` (GET) para una acción que cambia datos	`<form method="post">` + `{% csrf_token %}`
Corregí y sigue roto	Servidor colgado	`docker compose restart web` + `manage.py check`
Form guarda vacío	`clean_<campo>` sin `return`	Devolver el valor
Combos salen "Agotado" aunque tienen stock	Template compartido: la property solo existe en `CandyProduct` → en combos vale `""` y la comparación da falso	Usar algo que exista en los dos modelos (ver 2.13)
Producto con stock bajo sin botones de compra	El `elif` del cartel reemplazó al stepper	El cartel se suma, no reemplaza (ver 2.11)
Con stock == umbral sale "Agotado"	`<` en el template vs `<=` en la property	Usar la property booleana directamente
El form no deja guardar un valor que la consigna permite	`min_value` del form distinto a la consigna	Revisar `min_value` / `max_value` del campo del form
Un elemento nuevo queda apretado / rompe la fila	Fila flex pensada para menos elementos	`flex-wrap: wrap` + `flex-basis: 100%` + `order` (ver 2.14)
Cambié el CSS y no se ve	Caché del navegador	Ctrl+F5
`makemigrations`: *It is impossible to add a non-nullable field...*	Campo nuevo (fecha/número) sin `null=True` ni `default` en tabla con datos	Opción **2 (Quit)** → agregar `null=True, blank=True` (o `default=`) → `makemigrations` de nuevo
El form exige un campo que debería ser opcional	Tiene `null=True` pero falta `blank=True`	Agregar `blank=True` (+ `makemigrations` + `migrate`)
`Field.__init__() got an unexpected keyword`	Puse un campo (`forms.DateField`) donde va un widget	`forms.DateInput(attrs={...})` (ver 2.16)

Síntoma	Causa	Arreglo
`argument 'attrs'` (y el servidor se cae)		
El campo fecha se ve como texto libre, sin calendario	Widget por defecto = `type="text"`	`forms.DateInput(attrs={"type": "date"}, format="%Y-%m-%d")`
Al editar, la fecha aparece vacía	Falta `format` o está mal (`%M`/`%D` en vez de `%m`/`%d`)	`format="%Y-%m-%d"`
`'module' object is not callable`	`timezone().localdate()`	`timezone.localdate()`
`'>' not supported between instances of 'datetime.date' and 'datetime.timedelta'`	Comparé una fecha con una duración	Restar primero: `hoy - timedelta(days=6)` da una fecha
`'<' not supported between ... 'NoneType' and 'datetime.date'`	Campo opcional vacío (`None`) usado en una comparación	Chequear antes: `if not self.fecha: return False`
`TemplateSyntaxError` al poner una cuenta en el `{% if %}`	El template no ejecuta Python	Mover la cuenta a una `@property` del modelo
"Error during template rendering" con `Exception Location: .../models.py`	El error está en la property que usa el template, no en el template	Ir al archivo y línea de *Exception Location*
Docker: `request returned 500 Internal Server Error ... dockerDesktopLinuxEngine/_ping`	Docker Desktop trabado (no es el proyecto)	Reiniciar Docker Desktop; si sigue: cerrarlo, `wsl --shutdown`, abrirlo de nuevo
Tabla vacía ("No se encontró ninguna...") sin error, copiada de un listado	La vista es `TemplateView` → no existe `object_list`	Heredar de `ManagerListView` o cargar la lista en `get_context_data` (ver 2.17)
El buscador / "Por página" aparecen vacíos	`_list_controls.html` en una vista que no es `ManagerListView`	Heredar de `ManagerListView` o sacar el include

Síntoma	Causa	Arreglo
Se resalta otra sección del menú	`section` de la vista ≠ el del `{% if section == ... %}` del link	Poner el mismo texto en los dos
`FieldError: Cannot resolve keyword 'x'` al ordenar	Ordeno por un campo de `annotate` antes de crearlo	`annotate` primero, `order_by` después
Una columna muestra el dato de otra	Copié la `<td>` y no cambié la variable	Revisar cada `{{ movie.xxx }}` contra su `<th>`
Suma más chica de lo real	`Sum(..., distinct=True)`	`distinct` solo en `Count`
Totales en `None` / las filas sin ventas quedan **arriba** al ordenar desc	`Sum` sin filas = `NULL` y Postgres ordena los `NULL` primero	`Coalesce(Sum(...), 0)`
`Expression contains mixed types: IntegerField, DecimalField`	`Coalesce` con un 0 de otro tipo (ej: `Decimal("0")` sobre un entero)	Entero → `0`; decimal → `Decimal("0")`; `Count` sin `Coalesce`
Todo el sitio caído (localhost no responde)	`SyntaxError` en cualquier `.py` (ej: `import x as` sin nombre en `views/__init__.py`)	`docker compose logs web --tail 30` → última línea = archivo y línea
`NoReverseMatch ... with no arguments not found. 1 pattern(s) tried: [...<pk>...]`	La ruta pide un `pk` y no se lo pasé (o usé el `name` de otra ruta)	`{% url 'nombre' obj.pk %}` con el `name` de MI acción
`makemigrations` explota en `import_module`	Import circular: el `models.py` se importa a sí mismo	Borrar `from <misma app>.models import ...` de ese archivo
El modelo nuevo no aparece / `ForeignKey(Movie)` falla	Pegué la clase **adentro** de otra (con sangría)	Al final del archivo, `class X` en la columna 0
`relation "..." does not exist` justo después de `up -d`	El contenedor todavía estaba migrando	Esperar (`showmigrations` sin `[ ]`) y repetir

6. Checklist final de toda feature

- [ ] **Releer la consigna criterio por criterio** y tildar cada uno (es fácil olvidarse de la validación o del formato)
- [ ] ¿Toqué un modelo? → `makemigrations` + `migrate`
- [ ] ¿Campo opcional? → texto: `blank=True` / fecha o número: `null=True, blank=True`. Probar guardar con el campo vacío
- [ ] ¿El usuario carga el dato? → form (`Meta.fields`) + validación (`clean_` si hace falta)
- [ ] ¿Página/acción nueva? → vista + URL + template (+ link)
- [ ] ¿Se ve en el front? → template correcto, dato en el `render` de la vista correcta
- [ ] ¿Agregué una columna? → `<th>` + `<td>` + **contar columnas y actualizar el `colspan`** del `{% empty %}`
- [ ] ¿El dato está dentro de un `{% if %}/{% else %}`? → revisar **las dos ramas**
- [ ] ¿El template se usa para más de un modelo? → probar los dos (ej: productos y combos)
- [ ] ¿Quién puede usarlo? → mixin / `@login_required`
- [ ] ¿Cada usuario ve solo lo suyo? → `filter(user=request.user)`. Probar con dos cuentas
- [ ] ¿Totales/conteos? → filas en 0 (no `None`, no desaparecen), orden correcto
- [ ] ¿Vista nueva en una carpeta `views/`? → exportarla en `views/__init__.py`
- [ ] Probar manualmente el caso feliz **y** el caso de error **y** los bordes (valor exactamente igual al límite, vacío, 0)
- [ ] Correr los tests de la app que toqué

---

7. Estado de la práctica (para retomar en otra compu)

> **Para Claude:** estoy practicando para un parcial (implementar una feature simple en Butaca Cero, sin internet ni IA, **dura 1 hora** → features de ~40 min). Método: me das **solo la consigna** (historia de usuario + criterios de aceptación), yo exploro el código y lo escribo. Dame **pistas puntuales** cuando me trabo, **no la solución** (salvo que la pida explícitamente). Solo lo funcional, sin CSS ni estética. Al terminar cada feature, agregá a este machete lo que aprendí.

Código: las features 1–12 están en la rama `practica-parcial`. Desde la 13 practico sobre **`development`** (el código del parcial), sin commitear. Base limpia: `down -v` + `up -d` + `seed_practice` + `seed_demo --refresh-posters`.

Features terminadas:

1. Trailer de la película (URLField + botón en detalle)

2. Director (CharField + búsqueda con `Q`)
3. Filtro por género en el home (`request.GET`)
4. KPI "Entradas vendidas hoy" (`aggregate(Sum)`)
5. Idioma de la función (`TextChoices` + `get_..._display`)
6. Máximo de entradas por compra (config en `CinemaSettings` + validación en form)
7. Página "Funciones de hoy" para operador (vista + URL + template + permisos)
8. Reseñas de películas (modelo nuevo con FK + vista POST + `UniqueConstraint`)
9. Películas destacadas (toggle POST + `order_by` + marca en home) — hecha con el método de solo consigna
10. Stock bajo configurable en Candy (constante → `CinemaSettings` + property del modelo + template compartido productos/combos + arreglo de layout solo con CSS)
    - Pendiente de verificar: `test domain.candy` y el borde stock == umbral.

Casi terminada — 11. Aviso en la función (CharField opcional + `verbose_name` + marca en listado). Consigna:

1. Al crear/editar una función en el manager hay un campo opcional "Aviso para el público".
2. Máximo 80 caracteres. Si se carga, mínimo 5 (si no, error en el form).
3. Una función sin aviso se guarda sin problemas; las existentes quedan sin aviso.
4. En el detalle de la función (cliente), si tiene aviso se muestra destacado cerca de fecha/hora. Si no tiene, no aparece nada (ni caja vacía ni "None").
5. En el listado de funciones del manager, las que tienen aviso muestran una marca (ej: ⚠️ "Con aviso").

- [x] Campo `warning` en `Screening` + migración (`0008_screening_warning`)
- [x] `ScreeningForm`: en `Meta.fields` + label
- [x] Marca ⚠️ en `manager_funciones_list.html`
- [ ] Criterio 2: falta la validación del mínimo de 5 caracteres (`clean_warning`, ver 2.4)
- [ ] Criterio 4: en `screening_detail.html` el aviso está sin `{% if %}` → cuando no hay aviso queda un " · " colgando (ver sección 4)
- [ ] Probar: función con aviso, sin aviso, y aviso de 3 caracteres

Casi terminada — 12. Estrenos de la semana (DateField opcional + widget `type="date"` + property con fechas + marca en home). Consigna:

1. Al crear/editar una película en el manager hay un campo opcional "Fecha de estreno".
2. Una película sin fecha se guarda sin problemas; las existentes quedan sin fecha.
3. La fecha no puede ser posterior a dentro de un año (si no, error en el form).
4. En el home, las películas estrenadas en los últimos 7 días (hoy incluido) muestran "🎬 Estreno". Las más viejas, las futuras y las sin fecha, no.
5. El listado de películas del manager tiene una columna "Estreno" con formato `dd/mm/aaaa`; sin fecha muestra "—".

- [x] Campo `release_date` en `Movie` + migración (`0006_movie_release_date`)

- [x] `MovieForm`: en `Meta.fields`, label y widget `DateInput(type="date")`
- [x] Property en `Movie` con la cuenta de los 7 días + marca en `home.html`
- [x] Columna "Estreno" en `manager_peliculas_list.html` (con `colspan` actualizado)
- [ ] Criterio 2: probar crear una película **sin** fecha. ¿El form deja? (ver tabla `blank` vs `null` en 2.1)
- [ ] Criterio 3: falta la validación de "no más de un año" (`clean_release_date`, ver 2.4 y 2.15)
- [ ] Criterio 5: revisar el `{% if %}` de la celda (¿por qué campo pregunta?) y el formato de la fecha (ver sección 4)
- [ ] La property explota si se usa con una película sin fecha: hoy la salva el `{% if movie.release_date %}` de afuera en el template. Más seguro: chequear adentro de la property (ver 2.15)
- [ ] Probar: estreno hace 3 días, hace 20, hace exactamente 6 y 7, fecha futura, sin fecha, y fecha dentro de 2 años

13. Ranking de ventas por película (manager, sobre `development`) — `ManagerListView` + `annotate` (`Count` / `Sum` / `Coalesce`), ver 2.17.

- [x] Vista `RankingView(ManagerListView)` + URL + link en el menú + export en `views/__init__.py`
- [x] Columnas: funciones, entradas vendidas, recaudación (en 0 si no hay ventas)
- [x] Orden por funciones / por recaudación
- [ ] Orden por defecto: **entradas vendidas** de mayor a menor (la consigna lo pide así)
- [ ] Filtro desde / hasta por GET con `Sum(..., filter=Q(...))` (sin que desaparezcan las películas en 0)
- [ ] Sacar lo que sobra de la copia: `create_url_name`, `columns`; corregir "Recacudación" y el `data-label` de la columna de entradas

14. Películas favoritas (cliente, sobre `development`) — modelo usuario+película + toggle POST + página propia, ver 2.18.

- [x] Modelo `Favorite` + `UniqueConstraint` + migración (`0003_favorite`)
- [x] `toggle_favorite_view` (`@login_required` + `@require_POST`) + URL
- [x] Botón ♡/♥ en el detalle con `is_favorite`
- [x] `my_favorites` + `my_favorites.html` + link en el header
- [ ] Probar: marcar Dune y Wicked (Wicked primero), desmarcar Dune, otra cuenta ve lista vacía, duplicado rechazado por la base
- [ ] Extra: ♥ en el home sin una consulta por película

15. Idioma de la función — repetida SOLA, sin pistas (sobre `development`, ~30 min) — `TextChoices` en `Screening`, ver 2.2. ✅ Aprobada.

- [x] `Screening.Language` + campo `language` (`default` Subtitulada, `db_column="idioma"`) + migración `0008_screening_language`
- [x] `ScreeningForm`: `Meta.fields` + label
- [x] Columna en `manager_funciones_list.html` y en el chip del horario de `screening_detail.html`, con `get_language_display`
- [ ] `colspan="8"` → `9` en `manager_funciones_list.html` (ahora hay 9 columnas)
- [ ] Borrar `self.fields["language"].empty_label = ...` (no hace nada en un campo con `choices`)
- [ ] Mostrar el idioma también en el chip **Agotada** (rama `{% else %}`)

Ideas para después: Ficha de película en el manager (DetailView + agregaciones — más larga que 1 h), Exportar a CSV.